2-3 浮点数与IEEE754

1.实数的表示

1.1浮点数的表示格式

表达式

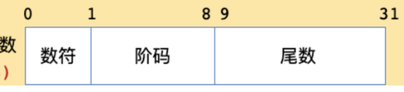
$$N=(-1)^S \times M \times R^E$$

- S取值为0或1，用来决定数X的符号
- M是一个二进制定点小数，称为数X的尾数，一般用原码表示
- E是一个二进制定点整数，称为数X的阶或指数，一般用移码表示
- R是基数，可以约定为2、4、16等

1.2IEEE754标准

float的23位尾数实际上提供了24位有效数字

32位IEEE754浮点数（c语言中的float）



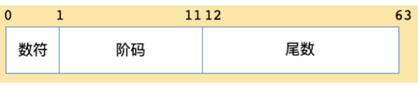
真值

$$(-1)^s \times 1.f \times 2^{e-127}$$

偏置值为127，E的取值为1～254

double的52位尾数实际上提供了53位有效数字

64位IEEE754浮点数（c语言中的double）



真值

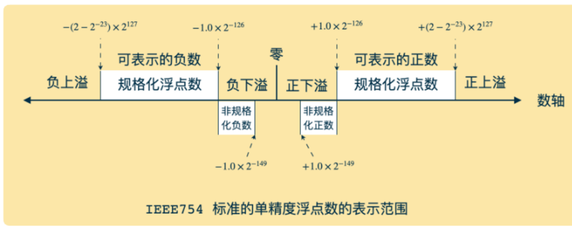
$$(-1)^s \times 1.f \times 2^{e-1023}$$

偏置值为1023，E的取值为1～2046

不同值的表示

值的类型	符号	阶码	尾数	值
正零	0	0	0	0
负零	1	0	0	0
正无穷大	0	255/2047	0	∞
负无穷大	1	255/2047	0	-∞
无定义数	-	255/2047	≠0	NaN (Not a Number)
非规格化数	-	0	f	$2^{-126} \times 0.f$ $2^{-1022} \times 0.f$

IEEE754标准的单精度浮点数的表示范围



IEEE754 标准的单精度浮点数的表示范围

1.3C语言中的浮点数类型

- int->float：不会发生溢出，可能有数据被舍入
- int/float->double：能保留精确值
- double->float：可能发生溢出，可能被舍入
- float/double->int：向0方向被截断

2.浮点数的加减运算

2.1对阶

定义：通过尾数移位使操作数的小数点位置对齐，从而表现出阶码相等

2.2尾数加减

定义：将对阶后的尾数按定点数加减运算规则进行运算

2.3尾数规格化

定义：上一步的尾数求和结束后可能得到不符合规格化的结果，此时要进行尾数左规或右规调整

运算法则：

- 右右规
  - 当结果为+/- 1x.x…x时，需要进行右规
  - 尾数右移1位，阶码+1
- 左左规
  - 当结果为+/- 0.0…01x…x时，需要进行左规
  - 尾数每左移1位，阶码-1，直到小数点前为1

2.4尾数的舍入处理

定义：在对阶和尾数右规时，可能会对尾数进行右移，为了保证运算精度，一般将低位移出的为保留下来，参加中间过程的运算，最后再将运算结果进行舍入，还原表示为IEEE754格式

类型：

- 就近舍入
- 朝+∞方向舍入——总是取右边最近可表示数，也称为正向舍入或朝上舍入
- 朝-∞方向舍入——总是取左边最近可表示数，也称为负向舍入或朝下舍入
- 朝0方向舍入——直接截取所需位数，丢弃后面所有位

2.5溢出判断

- 尾数溢出时可以通过右规进行调整，所以尾数溢出结果不一定溢出
- 左规/右规可能导致阶减小/增大，从而引发指数下溢/上溢，此时判定浮点数发生溢出，产生异常/按机器零处理
- 浮点数的溢出不是通过尾数溢出判断的，而是通过指数溢出判断的

3.数据的宽度和存储

3.1大小端

- 大端存储：和人类阅读顺序一致，数据低位放在高地址，数据高位放在低地址
- 小端存储：和人类阅读顺序相反，数据低位放在低地址，数据高位放在高地址
- 性质：一个编址单位内部的数据不受大小端影响，结构体内部的成员之间存放顺序不受大小端影响

3.2边界对齐

- 按x编址：存储器中每x位编一个地址，即一个地址中存储x位
- K字节大小的数据起始地址必须是K的整数倍

by joshuji 由 幕布 发布